Exercises

Exercise 4: Cleaning, summarising and joining data

Read Chapter 3 to help you complete the questions in this exercise.

This exercise picks up exactly where Exercise 3 left off and uses the same `cardiac` dataframe. If you are starting a fresh R session, either continue with the R script you wrote for Exercise 3 or run the two lines below before you go any further. They are the import from Exercise 3, Q5 and the `Fsex` factor from Exercise 3, Q7.

You will notice that `smoking` is still sitting there as an integer. That is deliberate. You will deal with it differently, and rather more elegantly, in Q9.

```
cardiac <- read.table('data/cardiacdata.txt', header = TRUE, sep = "\t",
                      na.strings = "NA", stringsAsFactors = TRUE)

cardiac$Fsex <- factor(cardiac$sex, levels = c(1, 2),
                       labels = c("Female", "Male"))
```

1. In Exercise 3 you learned how to pick out rows that meet a condition. You can use that skill to check whether your data are believable. This matters more than it sounds: a value that is simply wrong will not stop your code running, will not produce a warning, and will quietly change every result you calculate from it. Run `summary()` on your dataframe again and look at the minimum and maximum of each numeric variable. Three variables contain values that are not merely unusual but impossible. Two are impossibly low and one is impossibly high. Find them, work out which patient the impossibly high one belongs to, and then set all of the offending values to `NA` so they cannot contaminate anything you calculate later (you will need the square bracket `[ ]` notation from Section 3.4.2, this time on the left hand side of an assignment). Re-run `summary()` afterwards to check it worked. Why is `NA` the right answer here, rather than deleting the whole patient record or guessing what the value should have been?

2. Another useful way to manipulate your dataframes is to sort the rows based on the value of a variable (or combinations of variables). Rather counter-intuitively you should use the `order()` function to sort your dataframes, not the `sort()` function (see Section 3.4.3 of the Introduction to R book for an explanation). Ordering dataframes uses the same logic you practised in Q12 in Exercise 2. Let's practice with a straight forward example. Use the `order()` function to sort all rows in the `cardiac` dataframe in ascending order of systolic blood pressure (lowest to highest). Store this sorted dataframe in a variable called `cardiac_sys_sort`.

3. Now for something a little more complicated. Sort all rows in the `cardiac` dataframe by ascending order of systolic pressure within each level of smoking status. The trick here is to remember that you can order by more than one variable when using the `order()` function (see Section 3.4.3 again). Don't forget to assign your sorted dataframe to a new variable with a sensible name. Take a look at the bottom of the sorted dataframe: where have the patients with a missing smoking status ended up, and why?

4. Often, we would like to summarise variables by, for example, calculating a mean, median or counting the number of observations. To do this for a single variable it's fairly straight forward :

```
mean(cardiac$age)          # mean age
median(cardiac$systolic)   # median systolic blood pressure
length(cardiac$tchol)      # number of observations
```

4. (cont) Perhaps more interestingly, you might want to summarise one variable conditional on the level of another categorical variable. For example, write some R code to calculate the mean total cholesterol for each of the three smoking categories (see Section 3.5 of the Introduction to R book for a few hints). Next, calculate the median total cholesterol for each smoking category and for each sex.

5. Another useful function for summarising dataframes is `aggregate()` (see Section 3.5 again, or `?aggregate`). Use the `aggregate()` function to calculate the mean age, systolic pressure, diastolic pressure and total cholesterol for each smoking category (don't forget about those sneaky NA values). Next calculate the same means for each smoking category and each sex.

6. Knowing how many observations are present for each category (or combinations of categories) is useful to determine whether you have an adequate sample size (for subsequent modelling for example). Use the `table()` function to determine the number of patients in each smoking category (see Section 3.5 again for more information). Next use the same function to display the number of patients for each combination of smoking category and sex. Does `table()` tell you about the patients whose smoking status is missing?

7. Real analyses almost never involve a single file. The measurements you want are usually spread across several tables and you have to put them together first (see Section 3.4.5 of the Introduction to R book). The patients in this study were followed up ten years after their first examination, and those follow-up measurements are in a separate file. Download *'cardiac_followup.txt'* from the **Data** link, save it to your `data` directory and import it with `read.table()` into a variable called `followup`. How many rows does it have, and how does that compare with `cardiac`? Now use the `merge()` function to join the two together on the patient number, and assign the result to `cardiac_fu`. How many rows does the joined dataframe have, and can you explain why?

8. Losing 55 patients without being told is exactly the sort of thing that ruins an analysis, and it is why you check row counts every single time you join. Often what you actually want is to keep every patient from the first dataframe whether or not they have a match in the second. Look at Section 3.4.5 and the help file for `merge()`, and find the argument that does this. Redo the join keeping all 163 patients, assign it to `cardiac_all`, and check how many of them have a missing follow-up systolic pressure. Finally, try running `merge(cardiac, followup)` without the `by` argument at all. It works. Why is relying on that a bad habit?

9. There is one more join worth knowing, and it solves the `smoking` problem that has been hanging over you since Exercise 3. Rather than typing the labels out by hand as you did for `Fsex`, you can keep the codes and their meanings in their own small file and join them on. Download *'smoking_lookup.txt'* from the **Data** link, save it to your `data` directory and import it into a variable called `smoke_codes`. Take a look at it. Notice that the column holding the code is called `code`, not `smoking`, so the two dataframes have no column name in common and `by` on its own will not work. Look at Section 3.4.5 and the help file for `merge()` again and find the two arguments that let you name a different key in each dataframe. Join the labels onto `cardiac_all`, keeping all patients, and then check with `table()` that every patient has ended up with the right label.

10. Ok, we have spent quite a bit of time (and energy) learning how to clean, summarise and join dataframes. The last thing we need to cover is how to export a dataframe from R to an external file (see Section 3.6 of the book for more details). Export your cleaned and joined dataframe `cardiac_all` to a file called 'cardiac_clean.txt' in the `output` directory you created in Exercise 1. To do this you will need to use the `write.table()` function. You want to include the variable names in the first row of the file, but you don't want to include the row names. Also, make sure the file is a tab delimited file. Once you have created your file, try to open it in Microsoft Excel (or open source equivalent). Finally, and this is the part people forget: add a comment block at the top of your R script listing every change you made to the data and why. Which values did you set to `NA`, in which variables, and which patients did they belong to? Someone reading your work in six months, including you, needs to be able to answer that without re-running anything.

End of Exercise 4